Chapter 8: Automating Relationship Discovery

<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to identify the four bivariate relationship types (numeric-numeric, categorical-categorical, numeric-categorical, categorical-numeric) and select appropriate statistics for each
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to automate the computation of Pearson correlation, chi-square tests, and ANOVA F-statistics based on variable type combinations
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to generate targeted visualizations (scatterplots, bar charts, heatmaps) that match the relationship type being examined
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to build a controller function that integrates statistics and visualizations for all relationship types into a single automated workflow

8.1Introduction

Automating bivariate analyses (both statistics and charts) builds directly on the univariate automation techniques you learned previously. The key difference is that bivariate analysis focuses on *relationships* between variables rather than the distribution of a single variable. Instead of asking "What does this variable look like?", bivariate analysis asks "How does this variable relate to the outcome?"

In bivariate analysis, one variable typically plays the role of the *label* (also called the *target* or *outcome*), while the other is treated as a feature. Identifying the label correctly is essential, because it determines which analyses are meaningful and helps limit the number of unnecessary comparisons generated by automated functions.

In this book, bivariate statistics and visualizations are categorized based on the data type of the feature and the data type of the label. This allows you to treat bivariate analysis as a decision problem: once you know the data types involved, the appropriate statistical test and chart follow logically.

Specifically, there are four common bivariate relationship types: numeric–numeric, categorical–categorical, numeric–categorical, and categorical–numeric. A well-designed bivariate automation function uses this structure to determine whether to calculate Pearson's *r*, a chi-square (χ^2) test of independence, or an ANOVA (*F*-statistic), and to select an appropriate visualization for each case.

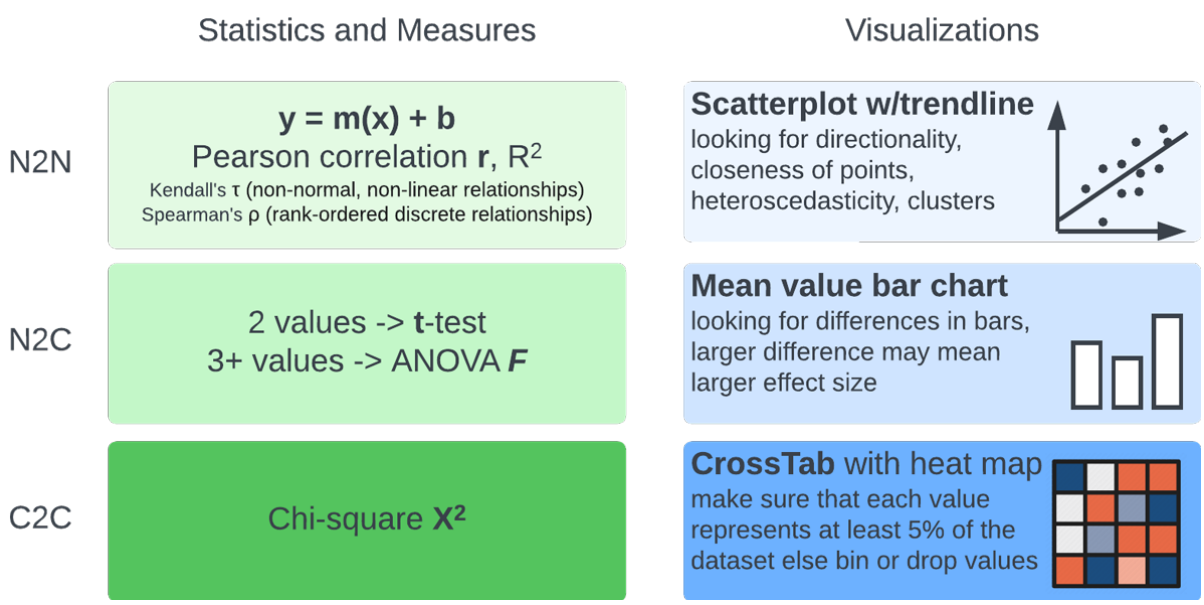


Figure 8.1: Summary of Bivariate Statistics and Charts by Data Type

In the prior chapter, you learned how to design functions that are:

- Automated
- Dynamic
- Error-resistant

We will assume those principles in this chapter and focus on applying them to relationship-driven analysis. Compared to univariate automation, bivariate automation requires more careful decision-making, because the choice of statistic and visualization must match *both* variables involved. In this

chapter, you will extend your automation skills by building functions that correctly identify relationship types, select appropriate tests and charts, and begin incorporating basic data validation and cleaning.

8.2Statistics Function

For this chapter, we're going to use the insurance.csv file we've used in the past as well as a new dataset on airline satisfaction that you can download below.

This download can be found online.

This video can be viewed online.

Let’s begin by briefly recalling the eight general steps involved in building an automation function:

1. Define the automation function
2. Import Python packages
3. Create variables needed for processing
4. Define the iteration (for example, a *for* loop)
5. Perform processing required for every iteration
6. Define the decision criterion
7. Perform processing required in each branch of the decision logic
8. Perform any final synthesis or cleanup

We followed these steps carefully in the prior chapter. They are listed here primarily as a reminder, because in this chapter we will move more quickly and focus on extending the logic rather than re-teaching the mechanics.

```
# Remember to mount Google Drive if needed
from google.colab import drive

drive.mount('/content/drive')
import pandas as pd

# Dataset with numeric label for testing
df_insurance = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/data/insurance.csv')

# Dataset with categorical label for testing
df_airline = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/data/airline_satisfaction.csv')
df_airline.head()

# Examine the output in your own notebook
```

Numeric-to-Numeric

We will start with numeric-to-numeric (N2N) relationships. When both the feature and the label are numeric, the appropriate primary statistic is the Pearson correlation coefficient (*r*). However, it is often useful to also calculate the regression line and the corresponding p-value, so we will compute all three.

Later in the course, we will extend this logic to include Kendall's tau (τ) and Spearman's rho (ρ) for cases where the assumptions of linear correlation are violated.

```
def bivariate_stats(df, label, roundto=4):
  import pandas as pd
  from scipy import stats
```

```
        output_df = pd.DataFrame(columns=['p', 'r', 'y = m(x) + b'])

        for feature in df.columns:
          if feature != label:
            if pd.api.types.is_numeric_dtype(df[feature]):
              m, b, r, p, err = stats.linregress(df[feature], df[label])
              output_df.loc[feature] = [round(p, roundto), round(r, roundto), f'y =
{round(m, roundto)}(x) + {round(b, roundto)}']

        return output_df
```

In this initial version of the function, we implemented several automation steps at once:

1. Defined the function and its inputs
2. Imported required packages
3. Created an output DataFrame to store results
4. Iterated through all features except the label
5. Applied a decision rule to isolate numeric features
6. Computed the appropriate statistics for N2N relationships

Next, let's test the function using the insurance dataset:

```
bivariate_stats(df_insurance, 'charges', 3)
```

	p	r	y = m(x) + b
age	0.000	0.299	y = 257.723(x) + 3165.885
bmi	0.000	0.198	y = 393.873(x) + 1192.937
children	0.013	0.068	y = 683.089(x) + 12522.496

Notice that statistics are produced only for numeric features. Categorical features are excluded because correlation cannot be computed on text values.

This initial version handles only numeric-to-numeric (N2N) relationships. Later in this chapter, we will extend the logic to also cover categorical-to-numeric (ANOVA) and categorical-to-categorical (chi-square) relationships, combine everything with automated visualizations, and add missing-data handling. For now, the important takeaway is the pattern: iterate through features, check data types, and compute the appropriate statistic.

Keep this version of bivariate_stats() handy—we will build on it shortly. The final version to save to functions.py will come at the end of the chapter when we construct the complete controller function.

8.3Visualization Functions

This video can be viewed online.

We are going to automate visualizations differently than statistics. Rather than creating one function that handles everything, we will create a separate function for each visualization type. This improves reusability—you can call each function independently in multiple contexts—and it reduces errors because each visualization's logic lives in exactly one place. The benefits will become clear as we build and reuse the functions below.

Once again, we will create functions for each relationship type: N2N, C2N/N2C, and C2C. Later, we will integrate them into a single controller function. The code below follows the same general approach as the video but includes several improvements.

We are going to work with the same datasets from the prior section. Read them into DataFrames if you have not already:

```
# Bring in some sample data
import pandas as pd

df_insurance = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/insurance.csv')
df_airline = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/airline_satisfaction.csv')
df_housing = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/housing.csv')
```

Numeric-to-Numeric (N2N): Scatterplots

Let's begin by creating a function that generates a scatterplot with a trendline. We could do this using either matplotlib or seaborn. If we used matplotlib, we would typically have to create the scatterplot and trendline separately and layer them together. But seaborn can create both in a single plot using *regplot*. At a minimum, *regplot* needs a dataset (or two vectors) and the names of the feature and label. We will also embed key statistics (regression equation, r, r^2, p, and n) directly in the chart so that you do not have to cross-reference the statistics table to interpret the relationship. Finally, we will make the function more reliable by handling missing data inside the function.

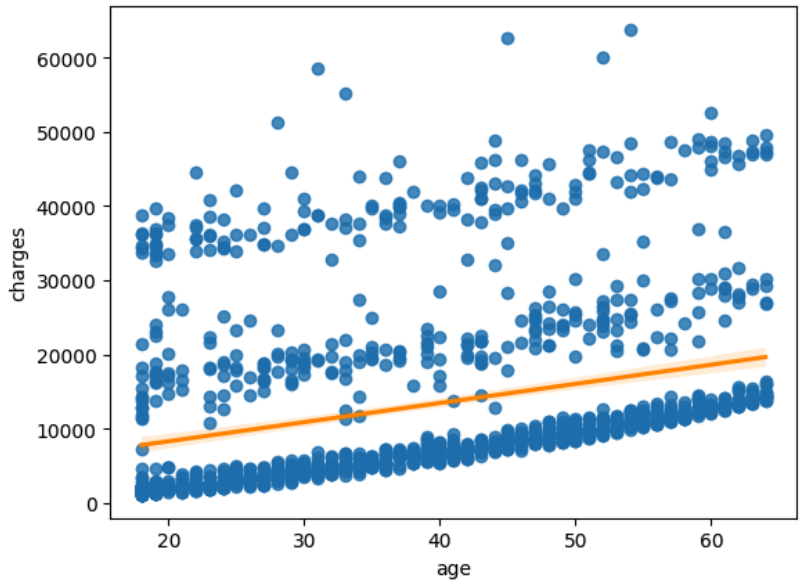
```
def scatterplot(df, feature, label, roundto=3, linecolor='darkorange', title=None, savepath=None, show=True):
  import pandas as pd
  from matplotlib import pyplot as plt
  import seaborn as sns
  from scipy import stats

  if feature == label:
    raise ValueError('feature and label must be different.')
  # Handle missing data
  df_temp = df[[feature, label]].dropna()
```

```
        if df_temp.shape[0] &lt; 3:
          raise ValueError('Not enough non-missing rows to plot and compute regression.')

        ax = plt.gca()
        # Create the plot (use data= signature for clarity and consistency)
        sns.regplot(data=df_temp, x=feature, y=label, ax=ax, line_kws={'color':
linecolor})
        # Calculate regression statistics
        m, b, r, p, err = stats.linregress(df_temp[feature], df_temp[label])
        # Build annotation text
        textstr  = 'Regression line:' + '\n'
        textstr += 'y = ' + str(round(m, roundto)) + 'x + ' + str(round(b, roundto)) +
'\n'
        textstr += 'r = ' + str(round(r, roundto)) + '\n'
        textstr += 'r2 = ' + str(round(r**2, roundto)) + '\n'
        textstr += 'p = ' + str(round(p, roundto)) + '\n'
        textstr += 'n = ' + str(df_temp.shape[0])
        # Optional title
        if title is None:
          title = str(feature) + ' vs. ' + str(label)

        ax.set_title(title)
        # Anchor text to axes so it stays stable across figure sizes
        ax.text(1.02, 0.02, textstr, fontsize=11, transform=ax.transAxes, va='bottom')
        # Optional save/show
        if savepath is not None:
          plt.savefig(savepath, bbox_inches='tight', dpi=200)

        if show:
          plt.show()

        return ax
```

Why not simply use *regplot* directly? Because it can be useful to calculate the same statistics from the prior section and embed them in the visualization. That way, you can interpret strength, direction, and statistical significance in the same place you view the chart.

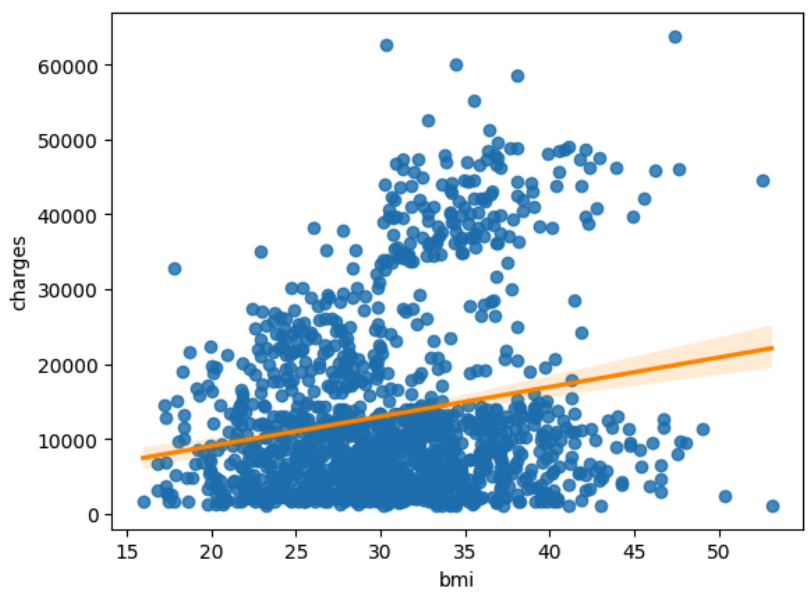
Next, let’s try this function out with three N2N relationships in the insurance dataset:

```
      scatterplot(df_insurance, 'age', 'charges')
```



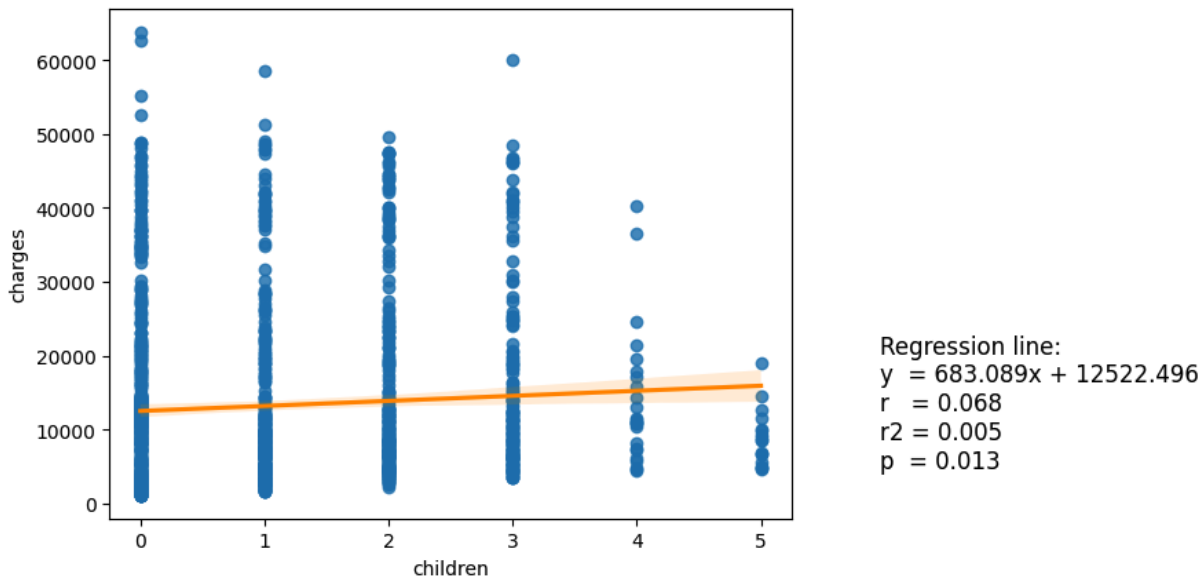
Regression line:
y = 257.723x + 3165.885
r = 0.299
r2 = 0.089
p = 0.0

```
scatterplot(df_insurance, 'bmi', 'charges')
```



Regression line:
y = 393.873x + 1192.937
r = 0.198
r2 = 0.039
p = 0.0

```
scatterplot(df_insurance, 'children', 'charges')
```



Next, we will apply the same overall idea to create a visualization for C2N/N2C relationships. Add scatterplot() to your functions.py file before moving on.

Categorical-to-Numeric (C2N/N2C): Bar charts

There are many options in seaborn for C2N/N2C relationships. The most basic and straightforward is a bar chart using *barplot*, so we will use that. Your goal is to create a function similar to the one above that embeds a one-way ANOVA F statistic and p-value in the chart. If you have trouble, examine the code below for help. In addition, we will make the function more robust by (1) handling missing data inside the function and (2) automatically swapping inputs so that the categorical variable always appears on the x-axis.

```
def bar_chart(df, feature, label, roundto=3, title=None, savepath=None, show=True):
  import pandas as pd
  from scipy import stats
  from matplotlib import pyplot as plt
  import seaborn as sns
```

```
        if feature == label:
          raise ValueError('feature and label must be different.')
        # Ensure we plot categorical on x-axis and numeric on y-axis
        is_num_feature = pd.api.types.is_numeric_dtype(df[feature])
        is_num_label = pd.api.types.is_numeric_dtype(df[label])

        if is_num_feature and not is_num_label:
          num = feature
          cat = label
        elif not is_num_feature and is_num_label:
          num = label
          cat = feature
        else:
          raise ValueError('bar_chart requires one numeric and one categorical variable.')
        # Handle missing data
        df_temp = df[[cat, num]].dropna()

        if df_temp[cat].nunique() &lt; 2:
          raise ValueError('Not enough categories to compare (need at least 2).')

        ax = plt.gca()
        # Create the plot (modern seaborn signature)
        sns.barplot(data=df_temp, x=cat, y=num, ax=ax, errorbar=None)
        # Create the group lists needed to calculate one-way ANOVA
        groups = list(df_temp[cat].unique())
        group_lists = [df_temp[df_temp[cat] == g][num] for g in groups]
        results = stats.f_oneway(*group_lists)
        F = results[0]
        p_anova = results[1]
        # Pairwise t-tests with Bonferroni correction
        ttests = []

        for i1, g1 in enumerate(groups):
          for i2, g2 in enumerate(groups):
            if i2 &gt; i1:
              vals_1 = df_temp[df_temp[cat] == g1][num]
              vals_2 = df_temp[df_temp[cat] == g2][num]
              t, p = stats.ttest_ind(vals_1, vals_2, equal_var=False, nan_policy='omit')
              ttests.append([str(g1) + ' - ' + str(g2), round(t, roundto), round(p,
roundto)])

        p_threshold = 0.05 / max(1, len(ttests))
        # Annotation text
        textstr  = 'ANOVA' + '\n'
        textstr += 'F: ' + str(round(F, roundto)) + '\n'
        textstr += 'p: ' + str(round(p_anova, roundto)) + '\n'
        textstr += 'n: ' + str(df_temp.shape[0]) + '\n\n'

        for ttest in ttests:
          if ttest[2] &lt;= p_threshold:
            if 'Sig. comparisons (Bonferroni)' not in textstr:
              textstr += 'Sig. comparisons (Bonferroni)' + '\n'

            textstr += str(ttest[0]) + ': t=' + str(ttest[1]) + ', p=' + str(ttest[2]) +
'\n'

        if title is None:
          title = str(num) + ' by ' + str(cat)

        ax.set_title(title)
        ax.text(1.02, 0.02, textstr, fontsize=10.5, transform=ax.transAxes, va='bottom')

        if savepath is not None:
```

```
    plt.savefig(savepath, bbox_inches='tight', dpi=200)

  if show:
    plt.show()

  return ax
```

In the *bar_chart()* function above, we created the visualization and embedded a one-way ANOVA F statistic and p-value. We also computed pairwise t-tests and used a Bonferroni-corrected threshold to reduce false positives when comparing multiple groups. Finally, the function automatically ensures the categorical variable is plotted on the x-axis, which makes the chart consistent even if the user passes the arguments in the opposite order.

One way to improve this function would be to bin rare categorical values (for example, values representing less than 5% of the dataset) into a category called *Other*. This is useful because it reduces visual clutter and reduces the number of t-tests for features with many values. We will come back to this idea later.

```
bar_chart(df_insurance, 'region', 'charges')
```

image

```
bar_chart(df_insurance, 'smoker', 'charges')
```

image

```
bar_chart(df_housing, 'Electrical', 'SalePrice')
```

image

In the example above, the housing dataset contains missing values for some categorical variables. The *bar_chart()* function handles this internally by dropping missing rows before calculating ANOVA and t-tests, so it works reliably across datasets without requiring separate preprocessing.

Save bar_chart() to functions.py.

Categorical-to-Categorical (C2C): Heat maps

For categorical-to-categorical relationships, we start by building a crosstab table of observed counts. Then we use that crosstab to compute a chi-square (X^2) test. Finally, we visualize the observed counts as a heat map. In this function, we also handle missing data and (optionally) bin rare categories into *Other* to keep the heat map readable.

The bin_categories() function you built in the data preparation chapter already handles this. Rather than redefining it, reuse the version in your functions.py file. That version accepts a list of features, so wrap a single feature name in a list when calling it:

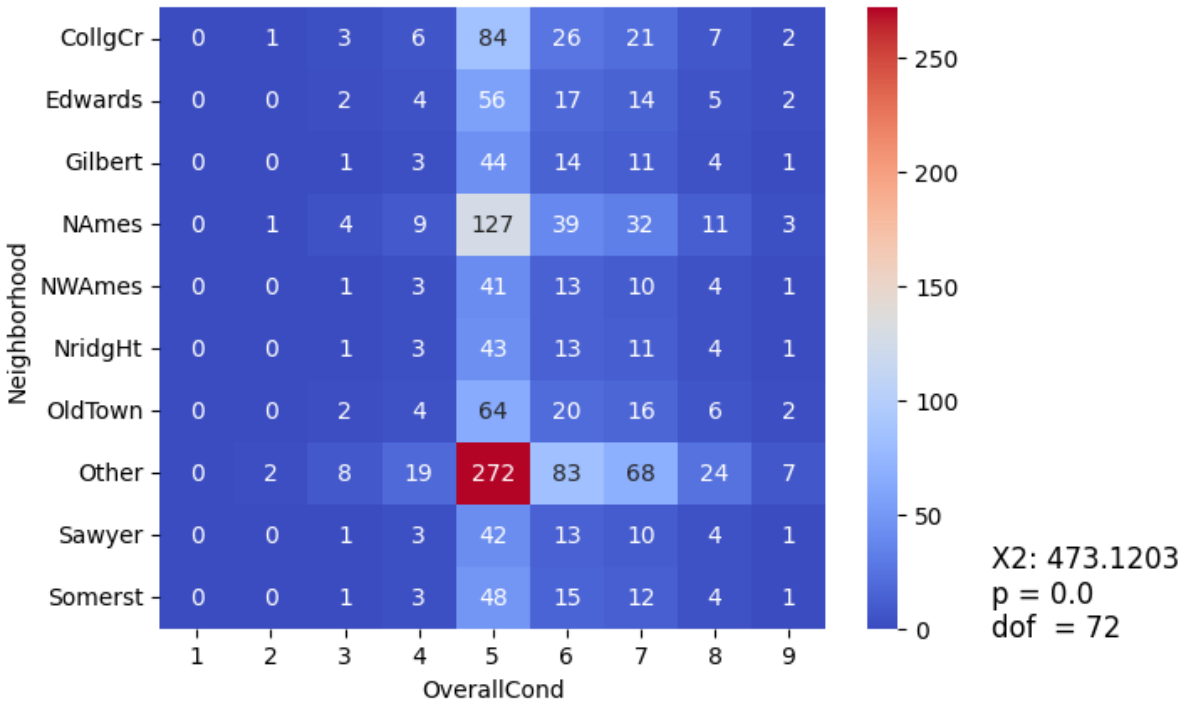
```
# Reuse bin_categories() from functions.py (already imported or pasted above)
# The multi-feature signature uses features=[] (a list):
df_housing = bin_categories(df_housing, features=['Neighborhood'])
```

Since all of these functions will eventually live in the same functions.py file, they can call each other directly without a module prefix. While you are developing in a notebook, either paste bin_categories() into the same cell or import it from functions.py.

```
# Print out a list of values before and after using the bin_categories function
print(df_housing['Neighborhood'].value_counts() / df_housing.shape[0], '\n\n')
df_housing = bin_categories(df_housing, features=['Neighborhood'])
print(df_housing['Neighborhood'].value_counts() / df_housing.shape[0])

# See output in your own notebook
```

```
def crosstab(df, feature, label, roundto=3, cutoff=0.05, title=None, savepath=None,
show=True):
  import pandas as pd
  from scipy.stats import chi2_contingency
  from matplotlib import pyplot as plt
  import seaborn as sns

  if feature == label:
    raise ValueError('feature and label must be different.')
  # Handle missing data
  df_temp = df[[feature, label]].dropna()

  if df_temp.shape[0] == 0:
    raise ValueError('No non-missing rows available for this pair of variables.')
  # Bin rare categories for readability (applies to feature by default)
  df_temp = bin_categories(df_temp, features=[feature], cutoff=cutoff)
  # Observed counts (what we typically want to visualize)
  ct = pd.crosstab(df_temp[feature], df_temp[label])
  # Chi-square based on observed counts
  X2, p, dof, expected = chi2_contingency(ct)
  ax = plt.gca()
  sns.heatmap(ct, annot=True, fmt='d', ax=ax)

  if title is None:
    title = str(feature) + ' vs. ' + str(label)

  ax.set_title(title)
  textstr  = 'X2: ' + str(round(X2, roundto)) + '\n'
  textstr += 'p: ' + str(round(p, roundto)) + '\n'
  textstr += 'dof: ' + str(dof) + '\n'
  textstr += 'n: ' + str(df_temp.shape[0])
  ax.text(1.02, 0.02, textstr, fontsize=10.5, transform=ax.transAxes, va='bottom')

  if savepath is not None:
    plt.savefig(savepath, bbox_inches='tight', dpi=200)

  if show:
    plt.show()

  return ax
```

```
crosstab(df_housing, 'Neighborhood', 'OverallCond')
```



Add crosstab() to functions.py.

8.4Controller Function

Now that we have a set of visualization helper functions to go with our bivariate statistics logic, we can combine everything into a single *controller* function. The controller's job is straightforward: iterate through each feature, identify the relationship type (N2N, C2N/N2C, or C2C), compute the correct statistics, and then call the matching visualization function. This is where the full bivariate statistics logic—including ANOVA for C2N/N2C and chi-square for C2C—comes together with the visualizations into one automated workflow.

The controller function depends on the helper functions you built earlier in this chapter (scatterplot(), bar_chart(), crosstab()) as well as bin_categories() from the data preparation chapter. Make sure all of these are defined in your notebook—either pasted directly or imported from functions.py—before running the controller.

```
def bivariate(df, label, roundto=4, viz=True):
  import pandas as pd
  from scipy import stats

  output_df = pd.DataFrame(
    columns=['missing', 'p', 'r', 'tau', 'rho', 'y = m(x) + b', 'F', 'X2', 'skew',
'unique', 'values']
  )

  for feature in df.columns:
    if feature != label:
      # Work with a temporary 2-column slice so missing-data handling is consistent
for every relationship
      df_temp = df[[feature, label]].dropna()
      missing = (df.shape[0] - df_temp.shape[0]) / df.shape[0]
      unique = df_temp[feature].nunique()
      # Bin categories on the TEMP slice (not the full df) to avoid mutating the
dataset globally
      if not pd.api.types.is_numeric_dtype(df_temp[feature]):
        df_temp = bin_categories(df_temp, features=[feature])

      if not pd.api.types.is_numeric_dtype(df_temp[label]):
        df_temp = bin_categories(df_temp, features=[label])
      # N2N: numeric label and numeric feature
      if pd.api.types.is_numeric_dtype(df_temp[feature]) and
pd.api.types.is_numeric_dtype(df_temp[label]):
        m, b, r, p, err = stats.linregress(df_temp[feature], df_temp[label])
        tau, tp = stats.kendalltau(df_temp[feature], df_temp[label])
        rho, rp = stats.spearmanr(df_temp[feature], df_temp[label])

        output_df.loc[feature] = [
          f'{missing:.2%}', round(p, roundto), round(r, roundto), round(tau,
roundto),
          round(rho, roundto), f'y = {round(m, roundto)}(x) + {round(b, roundto)}',
          '-', '-', df_temp[feature].skew(), unique, '-'
        ]

        if viz:
          scatterplot(df_temp, feature, label, roundto)  # Call the scatterplot
function
      # C2C: categorical label and categorical feature
      elif (not pd.api.types.is_numeric_dtype(df_temp[feature])) and (not
pd.api.types.is_numeric_dtype(df_temp[label])):
        contingency_table = pd.crosstab(df_temp[feature], df_temp[label])
        X2, p, dof, expected = stats.chi2_contingency(contingency_table)

        output_df.loc[feature] = [
```

```
                f'{missing:.2%}', round(p, roundto), '-', '-', '-', '-', '-', round(X2,
roundto),
                '-', unique, df_temp[feature].unique()
              ]

              if viz:
                crosstab(df_temp, feature, label, roundto)  # Call the crosstab function
            # C2N / N2C: one numeric, one categorical
            else:
              if pd.api.types.is_numeric_dtype(df_temp[feature]):
                skew = df_temp[feature].skew()
                num = feature
                cat = label
              else:
                skew = '-'
                num = label
                cat = feature

              groups = df_temp[cat].unique()
              group_lists = []

              for g in groups:
                g_list = df_temp[df_temp[cat] == g][num]
                group_lists.append(g_list)

              results = stats.f_oneway(*group_lists)
              F = results[0]
              p = results[1]

              output_df.loc[feature] = [
                f'{missing:.2%}', round(p, roundto), '-', '-', '-', '-', round(F,
roundto), '-', skew,
                unique, df_temp[cat].unique()
              ]

              if viz:
                bar_chart(df_temp, cat, num, roundto)  # Call the bar chart function

        return output_df.sort_values(by=['p'])
```

Notice what the controller function accomplishes: it applies the same missing-data handling for every feature, uses a simple decision tree to determine the relationship type, records the appropriate statistics in a single output table, and then calls the matching visualization helper function (scatterplot, bar chart, or heat map). The end result is a reusable bivariate EDA workflow you can run on almost any dataset with minimal effort.

When should you use bivariate() versus the individual functions? Use the controller when you want a broad overview of every feature in a dataset—it is designed for rapid, full-dataset exploration. Use the individual functions (scatterplot(), bar_chart(), crosstab()) when you want to examine a specific

relationship in detail, customize a single chart, or embed a visualization in a report. Both levels are useful, and having them as separate functions means you never have to choose one approach permanently.

Now let's test the controller with a few datasets:

```
bivariate(df_insurance, 'charges')
# See the output in your own notebook; very long
```

```
bivariate(df_airline, 'satisfaction')
# See the output in your own notebook; very long
```

```
bivariate(df_housing, 'SalePrice')
# See the output in your own notebook; very long
```

Add bivariate() to functions.py. You can also save the initial bivariate_stats() function if you want a lightweight option that returns only the statistics table without visualizations. At this point, your file should contain every reusable function from the past three chapters: unistats() from the EDA chapter, eight data preparation functions from the cleaning chapter, and the five bivariate functions you just built (bivariate_stats(), scatterplot(), bar_chart(), crosstab(), bivariate()). That single import line gives you a complete exploratory and preparation pipeline:

```
import sys
sys.path.append('/content/drive/MyDrive/Colab Notebooks/class/Predictive AI ML/In-
```

```
class notebooks')
      import functions as fun

      fun.bivariate(df_insurance, &quot;charges&quot;)
```

Reference: Complete functions.py
If you fell behind or want to compare your work, a reference version of functions.py containing all functions from these three chapters is available for download below. Use it as a fallback, but your own AI-improved versions may be better suited to your workflow and coding style.

8.5Practice

Next, see how you do with these practice problems to assess your ability to automate:

Practice 1: Improve N2N Statistics
The bivariate_stats function is a good start. But there are ways for us to improve it. This function does not tell us whether the statistics we used to analyze each relationship are truly valid. What does that mean? You might remember that the Pearson correlation coefficient depends on the assumption that both the feature and label are normally distributed. What if they aren't?

Modify the function to also calculate Kendall's tau (τ) (best for rank-ordered/ordinal data) and Spearman rho (ρ) correlations (best when the feature or label are not normally distributed or the relationship is non-linear). Add these columns next to Pearson r. To keep things simpler, do not worry about including the unique p-values for the tau and rho correlations. However, add two more columns at the end of the output_df: one with the skewness of each numeric feature and the other with a count of the number of unique values in that column. The analyst will need this to help her/him determine which correlation metric to use.

Test this function using the housing dataset to predict SalesPrice as the label. It should look similar to this:

	missing	p	r	τ	ρ	y = m(x) + b	F	X2	skew	unique
HeatingQC	0.00%	0.000	-	-	-	-	88.394	-	-	5
TotalBsmtSF	0.00%	0.000	0.614	0.435	0.603	y = 111.11(x) + 63430.629	-	-	1.524255	721
SaleType	0.00%	0.000	-	-	-	-	28.863	-	-	9
CentralAir	0.00%	0.000	-	-	-	-	98.305	-	-	2
...	...	...	...	...	...	...	...	...	...	...
MiscVal	0.00%	0.418	-0.021	-0.051	-0.063	y = -3.393(x) + 181068.755	-	-	24.476794	21
BsmtHalfBath	0.00%	0.520	-0.017	-0.01	-0.012	y = -5604.72(x) + 181243.659	-	-	4.103403	3
Utilities	0.00%	0.585	-	-	-	-	0.299	-	-	2
BsmtFinSF2	0.00%	0.664	-0.011	-0.031	-0.039	y = -5.603(x) + 181182.022	-	-	4.255261	144

79 rows × 10 columns

Practice 2: Improve Categorical Binning

If you have learned about data cleaning, then you might remember that categorical variables need to have a minimum representation for each categorical value (often 5%). For example, let's say you have a variable for 'Country' and there are 200 values that are distributed something like this: US = 75 rows, China = 75 rows, Brazil = 40 rows, Germany = 8 rows, Thailand = 1 row, Japan = 1 row. Three of those countries have value counts that comprise less than 5% of the overall dataset. Typically, we will bin those values into a bucket like 'Other' since it wouldn't be valid to draw conclusions from under-represented data.

The "5 percent" rule discussed above is just a rough guideline and not necessarily a strict rule. For example, what if you have very large datasets where 4 percent of the records is still 50 total records? The point is that it may be useful to have both a percent cutoff as well as a numeric count cutoff.

For example, maybe we only want to bin if the category group value is BOTH below 5 percent AND lower than 20 (or some other n) records.

Modify the bin_categories() function from the chapter to include this logic and test it using the housing dataset with the neighborhood feature. To do this, print out the value_counts() of the neighborhood feature before and after binning. But show both the count of values as well as the percent of values in the "before" version of neighborhood. It should look similar to this:

Neighborhood	Percentages	Counts
NAmes	0.154110	225
CollgCr	0.102740	150
OldTown	0.077397	113
Edwards	0.068493	100
Somerst	0.058904	86
Gilbert	0.054110	79
NridgHt	0.052740	77
Sawyer	0.050685	74
NWAmes	0.050000	73
SawyerW	0.040411	59
BrkSide	0.039726	58
Crawfor	0.034932	51
Mitchel	0.033562	49
NoRidge	0.028082	41
Timber	0.026027	38
IDOTRR	0.025342	37
ClearCr	0.019178	28
SWISU	0.017123	25
StoneBr	0.017123	25
MeadowV	0.011644	17
Blmngtn	0.011644	17
BrDale	0.010959	16
Veenker	0.007534	11
NPkVill	0.006164	9
Blueste	0.001370	2

Neighborhood	Percentages	Counts
Other	0.215753	315
NAmes	0.154110	225
CollgCr	0.102740	150
OldTown	0.077397	113
Edwards	0.068493	100
Somerst	0.058904	86
Gilbert	0.054110	79
NridgHt	0.052740	77
Sawyer	0.050685	74
NWAmes	0.050000	73
SawyerW	0.040411	59
BrkSide	0.039726	58
Crawfor	0.034932	51

Notice that more of the neighborhoods were kept (without being binned into "Other") with this modified function because they still have enough rows (> 50) even though their percentages are not > 0.05. Now you have a function that gives you greater control over binning.

Practice 3: Interpreting Effect Size Beyond Statistical Significance

In the bivariate and bivariate_stats functions developed in this chapter, statistical significance (p-values) is used as a proxy for identifying potentially important relationships. However, p-values alone do not measure the magnitude or practical importance of an effect.

Your task is to enhance the interpretation of bivariate results by incorporating effect size concepts alongside p-values. Specifically, consider how Pearson r, the ANOVA F statistic, and the chi-square statistic relate to effect size in numeric-to-numeric, categorical-to-numeric, and categorical-to-categorical relationships.

Create a short written summary explaining how an analyst should interpret each of the following together: p-value and Pearson r, p-value and F statistic, and p-value and chi-square statistic. Your explanation should clearly distinguish between statistical significance and practical significance.

Next, modify the output of the bivariate or bivariate_stats function to include a new column called *effect_size_note*. This column should contain a brief qualitative label such as “small effect,” “moderate effect,” or “large effect,” based on commonly accepted thresholds for each statistic.

You may need to research standard effect size guidelines (for example, Cohen’s conventions for r or chi-square). It is acceptable—and encouraged—to use an AI assistant to help identify reasonable thresholds and translate them into clear logic.

Test your enhanced function using the housing dataset with *SalePrice* as the label, and verify that the added column helps prioritize features that are both statistically significant and practically meaningful.

8.6Assignment

Complete the assignment below:

This assessment can be taken online.